

Министерство науки и высшего образования Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения высшего образования
**«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»**
(МИ ВлГУ)

Факультет _____ ИТР _____
Кафедра _____ ИС _____

ОТЧЕТ

по производственной (технологической (проектно-технологической))
практике

_____ (наименование практики)

ООО «Компания Инверсия»

_____ (место прохождения практики)

Тема Исследование и разработка веб-приложений на базе Spring
Framework и Spring Boot

Руководитель

_____ (оценка)

_____ Кульков Я. Ю.
(подпись) (от предприятия)

_____ Булаев А.В.
(подпись) (от института)

Студент _____ ИС-123
(группа)

_____ Зуев Е.И.
(фамилия, инициалы)

_____ (подпись) _____ (дата)

Содержание

Введение.....	4
1 Изучение принципов инверсии управления и внедрения зависимостей	5
2 Разработка собственного DI-контейнера.....	9
2.1 Архитектура и реализация контейнера.....	9
2.2 Демонстрация работы контейнера	12
3 Изучение Spring Context	13
3.1 XML-конфигурация	13
3.2 Java-конфигурация	14
3.3 Конфигурация на основе аннотаций	14
4 Разработка веб-приложения на Spring Boot	16
5 Интеграция базы данных и Spring Security.....	21
6 Разработка REST API.....	28
7 Результаты выполнения заданий	31
Заключение	37
Список используемых источников.....	40

					МИВУ.09.03.02-06.000 ПП		
Изм.	Лист	№ докум.	Подпись	Дата			
Разраб.		Зуев Е.И.			Исследование и разработка веб-приложений на базе Spring Framework и Spring Boot	Лит.	Лист
Провер.		Булаев А.В.					Листов
Конс.		Кульков Я.Ю.					3
Н. Контр.							40
Утверд.						МИВЛГУ ИС-123	

Введение

Современная разработка программного обеспечения предъявляет высокие требования к качеству кода, его тестируемости и поддерживаемости. Одним из ключевых принципов, обеспечивающих эти характеристики, является инверсия управления (Inversion of Control, IoC) – подход, при котором контроль над созданием и связыванием объектов передаётся от приложения внешнему контейнеру. Реализацией этого принципа служит внедрение зависимостей (Dependency Injection, DI), позволяющее создавать слабосвязанные, гибкие и легко тестируемые компоненты.

В экосистеме Java наиболее распространённой реализацией IoC-контейнера является Spring Framework. Для глубокого понимания принципов работы контейнеров необходимо не только изучить готовый фреймворк, но и реализовать собственный DI-контейнер с использованием аннотаций и рефлексии.

Целью данной работы является изучение принципов инверсии управления и внедрения зависимостей, разработка собственного DI-контейнера, исследование способов конфигурации Spring Context, разработка веб-приложения на Spring Boot с интеграцией базы данных, системой аутентификации и REST API.

Для достижения поставленной цели были решены следующие задачи:

- изучены понятия связанности кода, сильной и слабой связанности;
- исследован принцип инверсии управления и способы его реализации;
- изучена работа аннотаций в Java и механизма рефлексии;
- разработан собственный DI-контейнер AnnotationContainer;
- изучен Spring Context и реализованы три способа конфигурации;
- разработано веб-приложение на Spring Boot с CRUD-функциональностью;
- интегрирована база данных H2 с использованием JPA;
- добавлена система аутентификации и авторизации с Spring Security;
- реализован REST API с документацией Swagger UI.

1 Изучение принципов инверсии управления и внедрения зависимостей

Связанность кода (Coupling) – это степень зависимости между компонентами программной системы. Она определяет, насколько изменения в одном компоненте влияют на другие компоненты системы. В программной инженерии принято стремиться к слабой связанности, так как это повышает гибкость и переиспользуемость кода.

Сильная связанность (Tight Coupling) возникает в ситуациях, когда класс самостоятельно создаёт свои зависимости. Рассмотрим классический пример показанный на рисунке 1:

```
public class Auto {  
    private Engine engine = new Engine();  
    public void go() {  
        engine.startEngine();  
    }  
}
```

Рисунок 1 – Пример сильной связанности

В данном примере класс Auto жёстко привязан к конкретной реализации Engine. Это приводит к следующим проблемам:

- сложность тестирования – невозможно подменить реальную зависимость на mock-объект;
- сложность модификации – для замены двигателя необходимо изменить код класса;
- низкая переиспользуемость – класс нельзя использовать с другими реализациями;
- нарушение принципа инверсии зависимостей – модуль верхнего уровня зависит от модуля нижнего уровня.

Слабая связанность (Loose Coupling) достигается, когда класс получает свои зависимости извне, пример показан на рисунке 2. Для этого вводится абстракция (интерфейс), а конкретная реализация передаётся через конструктора, сеттер или параметры метода:

```
public interface Engine {
    void startEngine();
}

public class ElectricEngine implements Engine {
    @Override
    public void startEngine() { /* реализация */ }
}

public class Auto {
    private Engine engine;
    public Auto(Engine engine) {
        this.engine = engine;
    }
    public void go() {
        engine.startEngine();
    }
}
```

Рисунок 2 – Пример слабой связанности

Преимущества слабой связанности: упрощение тестирования (возможность подмены зависимостей на mock-объекты), гибкость замены реализаций без изменения кода класса, разделение ответственности и повышение переиспользуемости кода.

Инверсия управления (Inversion of Control, IoC) – это принцип, согласно которому управление созданием и связыванием объектов передаётся от приложения внешнему контейнеру. Вместо явного создания объектов через new, разработчик описывает зависимости, а контейнер сам создаёт объекты и связывает их.

IoC решает несколько ключевых проблем:

- устранение жёсткой связанности – компоненты зависят от абстракций, а не от конкретных реализаций;
- централизованное управление – все зависимости создаются и настраиваются в одном месте;
- конфигурируемость – поведение приложения может быть изменено без изменения кода;
- упрощение тестирования – легко подменять реализации для изолированного тестирования.

Внедрение зависимостей (Dependency Injection, DI) – это реализация IoC, при которой зависимости "внедряются" в объект извне, а не создаются им самим. Существуют три основных способа внедрения:

- Constructor Injection (внедрение через конструктор). Зависимости передаются через конструктор класса;
- Field Injection (внедрение через поля). Зависимости внедряются напрямую в поля с помощью аннотации `@Autowired`;
- Setter Injection (внедрение через сеттер). Зависимости передаются через метод-сеттер.

Аннотации – это метаданные, добавляемые к классам, методам и полям, пример аннотаций показан на рисунке 3. В DI-контейнере используются аннотации для маркировки компонентов и точек внедрения:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Component {
    String value() default "";
}

@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.FIELD, ElementType.CONSTRUCTOR})
public @interface Autowired {
}
```

Рисунок 3 – Пример аннотаций

Особенности аннотаций:

- @Retention (entionPolicy.RUNTIME) – аннотация доступна во время выполнения, что необходимо для работы Reflection;

- @Target – определяет, к каким элементам можно применять аннотацию.

Рефлексия (Reflection API) – механизм, позволяющий анализировать структуру классов во время выполнения. С его помощью можно получать информацию о классах, методах, полях, конструкторах, создавать экземпляры классов и вызывать методы динамически. В DI-контейнере Reflection используется для поиска классов с аннотацией @Component, поиска полей и конструкторов с @Autowired, создания экземпляров классов и внедрения зависимостей.

					МИВУ.09.03.02 – 06.000 ПП	Лист
						8
Изм.	Лист	№ докум.	Подпись	Дата		

2 Разработка собственного DI-контейнера

2.1 Архитектура и реализация контейнера

Разработанный DI-контейнер AnnotationContainer построен на следующих ключевых принципах: ленивая инициализация, внедрение через конструктора и поля, сканирование пакетов. Контейнер использует три основных хранилища, показанные на рисунке 4.

```
// Хранилище определений бинов (имя -> класс)
private final Map<String, Class<?>> beanDefinitions = new ConcurrentHashMap<>(); 5 usages

// Хранилище созданных экземпляров (имя -> экземпляр)
private final Map<String, Object> beanInstances = new ConcurrentHashMap<>(); 5 usages

// Кэш соответствия типа -> имя бина (для быстрого поиска по типу)
private final Map<Class<?>, String> typeToBeanName = new ConcurrentHashMap<>(); 7 usages
```

Рисунок 4 – Пример хранилищ

- beanDefinitions – хранит определения бинов (имя → класс);
- beanInstances – хранит созданные экземпляры (имя → объект);
- typeToBeanName – кэш соответствия типа → имя бина для быстрого поиска.

Регистрация компонентов показана на рисунке 5. Метод scan(String packageName) сканирует указанный пакет, находит все классы с аннотацией @Component и регистрирует их. Имя бина определяется из значения аннотации @Component.value() или генерируется из имени класса с маленькой буквы:


```

public void scan(String packageName) {
    String packagePath = packageName.replace('.', '/');
    URL resource = Thread.currentThread().getContextClassLoader().getResource(packagePath);
    if (resource == null) return;
    scanDirectory(new File(resource.getFile()), packageName);
}

private void registerBean(String name, Class<?> beanClass) {
    if (beanDefinitions.containsKey(name)) {
        throw new IllegalStateException("Бин '" + name + "' уже зарегистрирован");
    }
    beanDefinitions.put(name, beanClass);
}

```

Рисунок 5 – Регистрация компонентов

Ленивая инициализация показана на рисунке 6. Бины создаются не при регистрации, а только при первом запросе через метод `getBean()`. Это позволяет экономить ресурсы:

```

private Object getOrCreateBean(String name) { 1 usage  & ZuevEvgeny
    if (beanInstances.containsKey(name)) {
        logger.trace("Возврат существующего экземпляра бина '{}'", name);
        return beanInstances.get(name);
    }

    Class<?> beanClass = beanDefinitions.get(name);
    if (beanClass == null) {
        throw new NoSuchBeanException("Бин с именем '" + name + "' не зарегистрирован");
    }

    logger.info("Создание бина '{} (класс: {})", name, beanClass.getName());
    Object instance = createBeanInstance(beanClass);
    autowireFields(instance);

    beanInstances.put(name, instance);
    logger.info("Бин '{}' создан и инициализирован", name);

    return instance;
}

```

Рисунок 6 - Ленивая инициализация

Внедрение через конструктор показано на рисунке 7. Контейнер ищет конструктор с аннотацией `@Autowired` и разрешает зависимости для его параметров.

```
private Object createBeanWithAutowiredConstructor(Constructor<?> constructor) { 1 usage 2 ZuevEvgeny
    logger.trace("Создание экземпляра через @Autowired конструктор: {}", constructor.getName());

    Class<?>[] paramTypes = constructor.getParameterTypes();
    Object[] args = new Object[paramTypes.length];

    for (int i = 0; i < paramTypes.length; i++) {
        args[i] = getBean(paramTypes[i]);
        logger.trace("Параметр {}: внедрён бин типа {}", i, paramTypes[i].getName());
    }

    try {
        constructor.setAccessible(true);
        return constructor.newInstance(args);
    } catch (Exception e) {
        throw new BeanCreationException(
            "Не удалось создать экземпляр через конструктор " + constructor.getName(), e
        );
    }
}
```

Рисунок 7 – Внедрение через конструктор

Внедрение через поля показано на рисунке 8. После создания экземпляра контейнер обходит все поля и внедряет зависимости.

```
private void autowireFields(Object instance) { 1 usage 2 ZuevEvgeny
    Class<?> clazz = instance.getClass();
    logger.trace("Внедрение зависимостей в поля класса: {}", clazz.getName());

    // Рекурсивно обходим все поля класса, включая приватные из суперклассов
    while (clazz != null) {
        Field[] fields = clazz.getDeclaredFields();

        for (Field field : fields) {
            if (field.isAnnotationPresent(Autowired.class)) {
                logger.trace("Внедрение поля: {}", field.getName());

                Class<?> fieldType = field.getType();
                Object dependency = getBean(fieldType);

                try {
                    field.setAccessible(true);
                    field.set(instance, dependency);
                    logger.trace("Поле {} успешно внедрено", field.getName());
                } catch (IllegalAccessException e) {
                    throw new BeanCreationException(
                        "Не удалось внедрить поле " + field.getName() +
                        " в классе " + clazz.getName(), e
                    );
                }
            }
        }
        clazz = clazz.getSuperclass();
    }
}
```

Рисунок 8 – Внедрение через поля

2.2 Демонстрация работы контейнера

Для демонстрации работы были созданы три сервиса показанные на рисунке 9. Пример вывода работы контейнера показан на рисунке 10.

```
@Component
public class OrderService {
    public String getOrderDetails() {
        return "Детали заказа #12345";
    }
}

@Component
public class ShoppingCart {
    public void addItem(String item) {
        System.out.println("Товар добавлен в корзину: " + item);
    }
}

@Component
public class CheckoutService {
    @Autowired
    private OrderService orderService;
    @Autowired
    private ShoppingCart cart;

    public void checkout() {
        System.out.println(orderService.getOrderDetails());
        cart.addItem("Ноутбук");
        cart.addItem("Мышь");
        System.out.println("Заказ оформлен успешно!");
    }
}
```

Рисунок 9 – Демонстрация сервисов

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:D:\games\IntelliJ IDEA Community Edition 2025.2.1\lib\idea_rt.jar=61984" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8
=== Запуск DI-контейнера ===

Сканирование пакета...
2026-07-14 13:11:47.492 [main] INFO container.AnnotationContainer - Начало сканирования пакета: services
2026-07-14 13:11:47.499 [main] INFO container.AnnotationContainer - Регистрация бина: services.CheckoutService
2026-07-14 13:11:47.503 [main] INFO container.AnnotationContainer - Регистрация бина: services.OrderService
2026-07-14 13:11:47.503 [main] INFO container.AnnotationContainer - Регистрация бина: services.ShoppingCart
2026-07-14 13:11:47.504 [main] INFO container.AnnotationContainer - Завершено сканирование пакета: services. Найдено бинов: 3

Проверка наличия бинов:
Есть бин 'orderService'? true
Есть бин 'shoppingCart'? true
Есть бин 'checkoutService'? true

Получение бинов (ленивая инициализация):
2026-07-14 13:11:47.504 [main] INFO container.AnnotationContainer - Создание бина 'checkoutService' (класс: services.CheckoutService)
CheckoutService создан (#1)
2026-07-14 13:11:47.506 [main] INFO container.AnnotationContainer - Создание бина 'orderService' (класс: services.OrderService)
OrderService создан (#1)
2026-07-14 13:11:47.507 [main] INFO container.AnnotationContainer - Бин 'orderService' создан и инициализирован
2026-07-14 13:11:47.508 [main] INFO container.AnnotationContainer - Создание бина 'shoppingCart' (класс: services.ShoppingCart)
ShoppingCart создан (#1)
2026-07-14 13:11:47.509 [main] INFO container.AnnotationContainer - Бин 'shoppingCart' создан и инициализирован
2026-07-14 13:11:47.509 [main] INFO container.AnnotationContainer - Бин 'checkoutService' создан и инициализирован

Проверка, что бины - синглтоны:
checkout == checkout2? true
```

Рисунок 10 – Пример работы контейнера

3 Изучение Spring Context

3.1 XML-конфигурация

В XML-конфигурации все бины описываются в файле `appContext.xml`, пример листинга файла представлен на рисунке 11. Запуск приложения и вывод результата представлены на рисунке 12.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- Регистрация студентов -->
    <bean id="javaStudent" class="ru.kafpin.JavaStudent"/>
    <bean id="pythonStudent" class="ru.kafpin.PythonStudent"/>
    <bean id="cppStudent" class="ru.kafpin.CppStudent"/>

    <!-- Внедрение через конструктор (используем CppStudent) -->
    <bean id="departmentConstructor" class="ru.kafpin.Department">
        <constructor-arg ref="cppStudent"/>
    </bean>

    <!-- Внедрение через сеттер (используем PythonStudent) -->
    <bean id="departmentSetter" class="ru.kafpin.Department">
        <property name="student" ref="pythonStudent"/>
    </bean>

</beans>
```

Рисунок 11 – XML-конфигурация

```
try (ClassPathXmlApplicationContext context =
    new ClassPathXmlApplicationContext("appContext.xml")) {
    Department dept = context.getBean("department", Department.class);
    dept.displayInfo(); // Student: Petya, Can coding on: C++
}
}
```

а)

```
Student: Petya
Can coding on: C++
Student: Masha
Can coding on: Python

Process finished with exit code 0
```

б)

Рисунок 12 – Запуск приложения а), результат выполнения б)

					МИВУ.09.03.02 – 06.000 ПП	Лист
						13
Изм.	Лист	№ докум.	Подпись	Дата		

3.2 Java-конфигурация

В Java-конфигурации используется класс с аннотацией `@Configuration`, на рисунке 13 представлен пример класса с аннотацией.

```
@Configuration
@ComponentScan(basePackages = "ru.kafpin")
public class AppConfig {
    @Bean
    public Student misha() {
        return new JavaStudent("Misha");
    }
    @Bean
    public Student ivan() {
        return new JavaStudent("Ivan");
    }
}

@Component
public class Department {
    @Autowired
    public void setStudent(@Qualifier("ivan") Student student) {
        this.student = student;
    }
}
```

Рисунок 13 – Класс с аннотацией

3.3 Конфигурация на основе аннотаций

Классы помечаются аннотациями, а сканирование включается через `@ComponentScan`, пример представлен на рисунке 14.

```
@Component("bestStudent")
public class JavaStudent implements Student {
    private String name = "Misha";
}

@Component
public class Department {
    @Autowired
    @Qualifier("bestStudent")
    public void setStudent(Student student) {
        this.student = student;
    }
}
```

Рисунок 14 – Классы с аннотацией

Сравнительный анализ способов конфигурации представлен в таблице 1.

Таблица 1 – Анализ способов конфигурации

Характеристика	XML	Java-конфиг	Аннотации
Безопасность	Нет	Да	Да
Удобство использования	Низкое	Среднее	Высокое
Гибкость	Средняя	Высокая	Средняя

4 Разработка веб-приложения на Spring Boot

Проект создан через Spring Initializr (<https://start.spring.io>) с параметрами: Maven, Java 21, Spring Boot 3.2.0. Подключены зависимости: Spring Web, Thymeleaf, Bootstrap.

Структура проекта:

src/main/java/ru/kafpin/Task3/

└ SpringBootApplication.java

└ controller/

└ └ FormController.java

└ └ StudentController.java

└ model/Student.java

└ repository/StudentRepository.java

└ service/StudentService.java

src/main/resources/templates/

└ index.html, form.html, result.html

└ students.html, details.html, add-student.html

Модель Student содержит все необходимые поля в соответствии с заданием, представлены на рисунке 15.

```
package ru.kafpin.Task3.model;

public class Student {
    private Long id;
    private String firstName; // имя
    private String lastName; // фамилия
    private String middleName; // отчество
    private String email; // e-mail
    private String city; // город
    private String gender; // пол
    private int age; // возраст
    private String faculty; // факультет
    private String groupName; // группа
    private String login; // логин для портала

    // Конструкторы, геттеры и сеттеры

    public String getFullName() {
        return lastName + " " + firstName + " " + middleName;
    }
}
```

Рисунок 15 – Поля Student

Изм.	Лист	№ докум.	Подпись	Дата

Все поля имеют модификатор `private` и для каждого реализованы геттеры и сеттеры, что соответствует требованиям JavaBeans.

Сервисный слой обеспечивает доступ к данным из контроллеров и содержит бизнес-логику приложения. В данном случае сервис предоставляет методы для работы со студентами и вспомогательные методы для генерации группы и логина.

Листинг 1 – Сервисный слой (StudentService.java):

```
package ru.kafpin.Task3.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import ru.kafpin.Task3.model.Student;
import ru.kafpin.Task3.repository.StudentRepository;
import java.util.List;
import java.util.Optional;
@Service
public class StudentService {
    private final StudentRepository studentRepository;
    @Autowired
    public StudentService(StudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }
    public List<Student> getAllStudents() {
        return studentRepository.getStudents();
    }
    public Optional<Student> getStudentById(Long id) {
        return studentRepository.getStudentById(id);
    }
    public Student saveStudent(Student student) {
        return studentRepository.save(student);
    }
    public void deleteStudentById(Long id) {
        studentRepository.deleteById(id);
    }
    public boolean existsById(Long id) {
        return studentRepository.existsById(id);
    }

    public static String generateGroupName(int enrollmentYear) {
        String lastTwoDigits = String.valueOf(enrollmentYear % 100);
        return "ПИНЗ-" + lastTwoDigits;
    }

    public static String generateLogin(String groupName, Long id) {
        return "student-" + groupName.toLowerCase() + "-" + id;
    }
}
```


Аннотация `@Service` указывает Spring, что этот класс является сервисным компонентом. В конструкторе используется `@Autowired` для внедрения зависимости `StudentRepository`. Репозиторий хранит данные в памяти и предоставляет методы для CRUD-операций. Инициализация начальными данными выполняется с помощью `@PostConstruct`.

Листинг 2 – Реализация репозитория (StudentRepository.java):

```
package ru.kafpin.Task3.repository;
import jakarta.annotation.PostConstruct;
import org.springframework.stereotype.Repository;
import ru.kafpin.Task3.model.Student;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
@Repository
public class StudentRepository {
    private List<Student> students;
    private Long currentId = 5L;
    @PostConstruct
    public void init() {
students = new ArrayList<>();
students.add(new Student(1L, "Михаил", "Иванов", "Петрович",
    "misha@mail.ru", "Москва", "Мужской", 20,
    "Радиотехнический", "РИ-120", "student-ri120-1"));
students.add(new Student(2L, "Мария", "Петрова", "Сергеевна",
    "masha@mail.ru", "Санкт-Петербург", "Женский", 19,
    "Информационных технологий", "ПИНз-120", "student-pinz120-2"));
students.add(new Student(3L, "Магомед", "Алиев", "Расулович",
    "magomed@mail.ru", "Махачкала", "Мужской", 20,
    "Радиотехнический", "РИ-121", "student-ri121-3"));
students.add(new Student(4L, "Анна", "Смирнова", "Алексеевна",
    "anna@mail.ru", "Владимир", "Женский", 18,
    "Информационных технологий", "ПИНз-121", "student-pinz121-4"));
    }

    public List<Student> getStudents() {
        return students;
    }

    public Optional<Student> getStudentById(Long id) {
        return students.stream()
            .filter(s -> s.getId().equals(id))
            .findFirst();
    }

    public Student save(Student student) {
        if (student.getId() == null) {
            student.setId(currentId++);
        } else {
```

```

        deleteById(student.getId());
    }
    students.add(student);
    return student;
}

public void deleteById(Long id) {
    students.removeIf(s -> s.getId().equals(id));
}

public boolean existsById(Long id) {
    return students.stream().anyMatch(s.getId().equals(id));
}
}

```

Аннотация `@Repository` указывает Spring, что этот класс является репозиторием. Метод `@PostConstruct` автоматически вызывается после создания экземпляра для инициализации начальными данными.

В подходе Spring к построению web-приложений HTTP-запросы обрабатываются контроллерами. Компоненты-контроллеры идентифицируются аннотацией `@Controller`. Пример контроллера из задания 3 представлен в листинге 3.

Листинг 3 – Пример одного из контроллеров (StudentController.java):

```

package ru.kafpin.Task3.controller;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import ru.kafpin.Task3.model.Student;
import ru.kafpin.Task3.service.StudentService;
import java.time.Year;
@Controller
public class FormController {
    private final StudentService studentService;

    @Autowired
    public FormController(StudentService studentService) {
        this.studentService = studentService;
    }
    /**
     * Отображает форму для добавления студента.
     * Создает пустой объект Student и передает его в шаблон.
     */
    @GetMapping("/form")
    public String showForm(Model model) {

```

```

        model.addAttribute("student", new Student());
        return "form";
    }

    /**
     * Обработывает отправку формы.
     * Автоматически генерирует группу и логин.
     */
    @PostMapping("/form")
    public String submitForm(@ModelAttribute Student student, Model
model) {
        // Получаем текущий год
        int currentYear = Year.now().getValue();

        // Генерируем группу по году поступления
        String groupName = StudentService.generateGroupName(cur-
rentYear);
        student.setGroupName(groupName);

        // Сохраняем студента, чтобы получить ID
        Student saved = studentService.saveStudent(student);

        // Генерируем логин
        String login = StudentService.generateLogin(groupName,
saved.getId());
        saved.setLogin(login);
        studentService.saveStudent(saved);

        model.addAttribute("student", saved);
        return "result";
    }
}

```

5 Интеграция базы данных и Spring Security

Для работы с базой данных добавлены зависимости в pom.xml. H2 – это встраиваемая реляционная база данных, которая запускается вместе с приложением и не требует отдельной установки. В режиме in-memory база данных создаётся при старте приложения и полностью уничтожается при остановке, что удобно для разработки и отладки. На рисунке 17 представлен фрагмент кода файла xml.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

Рисунок 17 – Фрагмент кода XML

Настройки базы данных (application.properties) представлены на рисунке 18.

```
# H2 Database
spring.datasource.url=jdbc:h2:mem:institut
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password

# H2 Console (доступна по адресу /h2-console)
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# JPA
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Инициализация данных
spring.sql.init.mode=always
spring.jpa.defer-datasource-initialization=true
```

Рисунок 18 – Настройка базы данных

JPA (Java Persistence API) – это спецификация для работы с реляционными базами данных в Java. Сущность – это POJO-класс, который отображается на таблицу в базе данных. Для этого используются аннотации из пакета jakarta.persistence. Сущность Student с аннотациями JPA и валидации представлена в листинге 4.

Листинг 4 – Сущность Student с аннотациями JPA (PlanEditor.java):

```
package ru.kafpin.Task3.model;
import jakarta.persistence.*;
import jakarta.validation.constraints.*;
@Entity
@Table(name = "students")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Long id;
    @NotBlank(message = "Имя обязательно")
    @Size(min = 2, max = 30, message = "Имя должно содержать от 2 до
30 символов")
    @Column(name = "first_name", nullable = false)
    private String firstName;
    @NotBlank(message = "Фамилия обязательна")
    @Size(min = 2, max = 30)
    @Column(name = "last_name", nullable = false)
    private String lastName;
    @Column(name = "middle_name")
    private String middleName;
    @Email(message = "Некорректный формат email")
    @NotBlank(message = "Email обязателен")
    @Column(name = "email", unique = true, nullable = false)    //
Уникальное поле
    private String email;

    @NotBlank(message = "Город обязателен")
    @Column(name = "city", nullable = false)
    private String city;

    @NotBlank(message = "Пол обязателен")
    @Column(name = "gender", nullable = false)
    private String gender;

    @Min(value = 16, message = "Возраст должен быть не менее 16 лет")
    @Max(value = 60, message = "Возраст должен быть не более 60 лет")
    @Column(name = "age", nullable = false)
    private int age;

    @NotBlank(message = "Факультет обязателен")
    @Column(name = "faculty", nullable = false)
```

```

        private String faculty;

        @Column(name = "group_name")
        private String groupName;

        @Column(name = "login")
        private String login;

        // Конструкторы, геттеры и сеттеры
        public String getFullName() {
            return lastName + " " + firstName + " " + (middleName !=
null ? middleName : "");
        }
    }

```

Spring Data JPA предоставляет мощный механизм для работы с базами данных через репозитории. Репозиторий – это интерфейс, который наследуется от одного из базовых интерфейсов Spring Data, таких как CrudRepository или JpaRepository. Spring автоматически генерирует реализацию этого интерфейса во время выполнения, что избавляет от необходимости писать boilerplate-код для CRUD-операций. В листинге 5 представлена реализация класса репозитория.

Листинг 5 – Spring Data JPA (StudentRepository.java):

```

package ru.kafpin.Task3.repository;
import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;
import ru.kafpin.Task3.model.Student;
import java.util.List;
import java.util.Optional;
@Repository
public interface StudentRepository extends CrudRepository<Student,
Long> {
    // Базовые методы наследуются от CrudRepository:
    // - save()           – сохранить сущность
    // - findById()       – найти по ID
    // - findAll()        – найти все
    // - deleteById()     – удалить по ID
    // - existsById()     – проверить существование
    // Дополнительные методы поиска (Spring Data JPA генерирует ре-
ализацию автоматически)

    List<Student> findAll(); // Переопределяем для возврата List
вместо Iterable

    Optional<Student> findById(Long id);

    void deleteById(Long id);

```

```

        boolean existsById(Long id);

        // Поиск по фамилии (частичное совпадение, без учета регистра)
        List<Student> findByLastNameContainingIgnoreCase(String lastName);

        // Поиск по факультету (точное совпадение, без учета регистра)
        List<Student> findByFacultyIgnoreCase(String faculty);

        // Поиск по группе (без учета регистра)
        List<Student> findByGroupNameIgnoreCase(String groupName);

        // Поиск по городу (без учета регистра)
        List<Student> findByCityIgnoreCase(String city);

        // Поиск по диапазону возраста
        List<Student> findByAgeBetween(int minAge, int maxAge);
    }

```

Spring Security – это мощный фреймворк для обеспечения безопасности приложений. Он предоставляет механизмы аутентификации (проверка подлинности пользователя) и авторизации (проверка прав доступа). Как только в pom.xml добавляется зависимость spring-boot-starter-security, Spring Boot автоматически защищает все HTTP-эндпоинты с помощью базовой аутентификации. Добавление зависимости представлено на рисунке 19.

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
    <groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>

```

Рисунок 19 – Зависимости Spring Security

Для хранения пользователей в базе данных создаются две сущности: User (реализует UserDetails) и Role (реализует GrantedAuthority). Это позволяет Spring

Security загружать информацию о пользователе из БД. Сущности представлены на рисунках 20-21.

```
package ru.kafpin.Task3.model;

import org.springframework.security.core.GrantedAuthority;

public enum Role implements GrantedAuthority {
    ROLE_USER, // Обычный пользователь (только просмотр)
    ROLE_ADMIN; // Администратор (полный доступ)

    @Override
    public String getAuthority() {
        return name(); // Возвращает "ROLE_USER" или "ROLE_ADMIN"
    }
}
```

Рисунок 20 - Enum Role

```
package ru.kafpin.Task3.model;

import jakarta.persistence.*;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.util.Collection;
import java.util.List;

@Entity
@Table(name = "users")
public class User implements UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String password;

    @Enumerated(EnumType.STRING) // Хранит название роли как строку в БД
    @Column(nullable = false)
    private Role role;
```

Рисунок 21 - Сущность User

UserService реализует интерфейс UserDetailsService, который Spring Security использует для загрузки пользователя по имени. В методе loadUserByUsername() выполняется поиск пользователя в репозитории. Если пользователь найден, его данные оборачиваются в объект org.springframework.security.core.userdetails.User, который Spring Security использует для аутентификации.

					МИВУ.09.03.02 – 06.000 ПП	Лист
						25
Изм.	Лист	№ докум.	Подпись	Дата		

Листинг 6 – Реализация интерфейса UserDetailsService (UserService.java):

```
package ru.kafpin.Task3.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import ru.kafpin.Task3.model.Role;
import ru.kafpin.Task3.model.User;
import ru.kafpin.Task3.repository.UserRepository;
import java.util.List;
@Service
public class UserService implements UserDetailsService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    @Autowired
    public UserService(UserRepository userRepository, PasswordEncoder passwordEncoder) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    /**
     * Загружает пользователя по имени для Spring Security.
     * Выбрасывает UsernameNotFoundException, если пользователь не найден.
     */
    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("Пользователь не найден: " + username));

        // Возвращаем объект UserDetails, который Spring Security использует для аутентификации
        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPassword(),
            List.of(new SimpleGrantedAuthority(user.getRole().getAuthority()))
        );
    }

    /**
     * Сохраняет пользователя с шифрованием пароля.
     */
    public User saveUser(User user) {
        user.setPassword(passwordEncoder.encode(user.getPassword()));
        return userRepository.save(user);
    }

    /**
     * Создаёт нового пользователя с указанной ролью.
     */
}
```

						Лист
					МИВУ.09.03.02 – 06.000 ПП	26
Изм.	Лист	№ докум.	Подпись	Дата		

```

    */
    public User createUser(String username, String rawPassword, Role role)
    {
        User user = new User();
        user.setUsername(username);
        user.setPassword(rawPassword); // пароль зашифруется в saveUser()
        user.setRole(role);
        return saveUser(user);
    }
    /**
     * Проверяет, существует ли пользователь с таким именем.
     */
    public boolean userExists(String username) {
        return userRepository.findByUsername(username).isPresent();
    }
}

```

SecurityConfig – это основной класс конфигурации безопасности. Он определяет правила доступа к различным URL, настройки формы входа и выхода, а также механизм шифрования паролей. Фрагмент кода показан на рисунке 22.

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity // Включает аннотации @PreAuthorize для контроля доступа на уровне методов
public class SecurityConfig {

    /**
     * Основная цепочка фильтров безопасности.
     * Определяет правила доступа к URL, форму входа и выхода.
     */
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            // Настройка авторизации запросов
            .authorizeHttpRequests(auth -> auth
                // Эти URL доступны всем без авторизации
                .requestMatchers("/", "/login", "/register", "/h2-console/**").permitAll()
                // Все запросы к /students/** требуют аутентификации
                .requestMatchers("/students/**").authenticated()
                // Любые другие запросы требуют аутентификации
                .anyRequest().authenticated()
            )
            // Настройка формы входа
            .formLogin(form -> form
                .loginPage("/login") // Своя страница входа
                .defaultSuccessUrl("/students", true) // Перенаправление после успешного входа
                .failureUrl("/login?error=true") // URL при ошибке входа
                .permitAll()
            )
        )
    }
}

```

Рисунок 22 - Класс конфигурации безопасности

6 Разработка REST API

REST (Representational State Transfer) – это архитектурный стиль для построения распределённых систем. Основные принципы REST:

- клиент-серверная архитектура – разделение ответственности между клиентом и сервером;
- отсутствие состояния – каждый запрос содержит всю необходимую информацию для его обработки;
- кэшируемость – ответы сервера могут быть кэшированы клиентом;
- единообразие интерфейса – использование стандартных HTTP-методов;
- слоистая система – возможность промежуточных серверов.

В Spring Framework для создания REST API используются контроллеры с аннотацией `@RestController`, которая является комбинацией `@Controller` и `@ResponseBody`. Это означает, что каждый метод контроллера по умолчанию возвращает данные (в формате JSON), а не представление (HTML-страницу). Аннотация `@RequestMapping` позволяет задать общий префикс для всех эндпоинтов контроллера. Пример показан на рисунке 23.

```
@RestController
@RequestMapping("/api/students")
@Tag(name = "Student API", description = "CRUD операции со студентами")
public class StudentController {
    // Методы контроллера
}
```

Рисунок 23 – Фрагмент класса контроллера

Полный набор CRUD-операций (Create, Read, Update, Delete) реализуется с помощью четырёх основных HTTP-методов.

Операция чтения (GET) используется для получения данных показана на рисунке 24. Для получения списка всех студентов используется эндпоинт без

параметров, а для получения конкретного студента – с идентификатором в пути.

```
@GetMapping
public List<Student> getAllStudents() {
    return studentRepository.findAll();
}

@GetMapping("/{id}")
public Student getStudentById(@PathVariable Long id) {
    return studentRepository.findById(id)
        .orElseThrow(() -> new EntityNotFoundException("Студент с ID " + id + " не найден"));
}
```

Рисунок 24 – Операция чтения GET

Операция создания (POST) позволяет создать новый ресурс, пример показан на рисунке 25. Клиент отправляет JSON-объект с данными в теле запроса. Аннотация `@RequestBody` преобразует JSON в Java-объект, а `@Valid` запускает валидацию. Успешное создание возвращает статус 201 Created.

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Student createStudent(@Valid @RequestBody StudentDTO studentDTO) {
    Department department = departmentRepository.findById(studentDTO.getDepartmentId())
        .orElseThrow(() -> new EntityNotFoundException("Кафедра не найдена"));

    Student student = new Student();
    student.setName(studentDTO.getName());
    student.setAge(studentDTO.getAge());
    student.setDepartment(department);

    return studentRepository.save(student);
}
```

Рисунок 25 – Операция создания POST

Операция обновления (PUT) полностью заменяет существующий ресурс новыми данными, пример показан на рисунке 26. Сначала проверяется существование ресурса, затем обновляются все поля.

```

@PutMapping("/{id}")
public Student updateStudent(@PathVariable Long id, @RequestBody StudentDTO studentDTO) {
    Student student = studentRepository.findById(id)
        .orElseThrow(() -> new EntityNotFoundException("Студент с ID " + id + " не найден"));

    student.setName(studentDTO.getName());
    student.setAge(studentDTO.getAge());
    student.setDepartment(departmentRepository.findById(studentDTO.getDepartmentId())
        .orElseThrow(() -> new EntityNotFoundException("Кафедра не найдена")));

    return studentRepository.save(student);
}

```

Рисунок 26 – Операция обновления PUT

Операция удаления (DELETE) удаляет ресурс по идентификатору, пример показан на рисунке 27. Успешное удаление возвращает статус 204 No Content.

```

@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void deleteStudent(@PathVariable Long id) {
    if (!studentRepository.existsById(id)) {
        throw new EntityNotFoundException("Студент с ID " + id + " не найден");
    }
    studentRepository.deleteById(id);
}

```

Рисунок 27 – Операция удаления DELETE

7 Результаты выполнения заданий

Разработанный REST API был протестирован с использованием инструментов Postman и Swagger UI. Все методы работают корректно и возвращают ожидаемые результаты, представленные на рисунках 28-31.

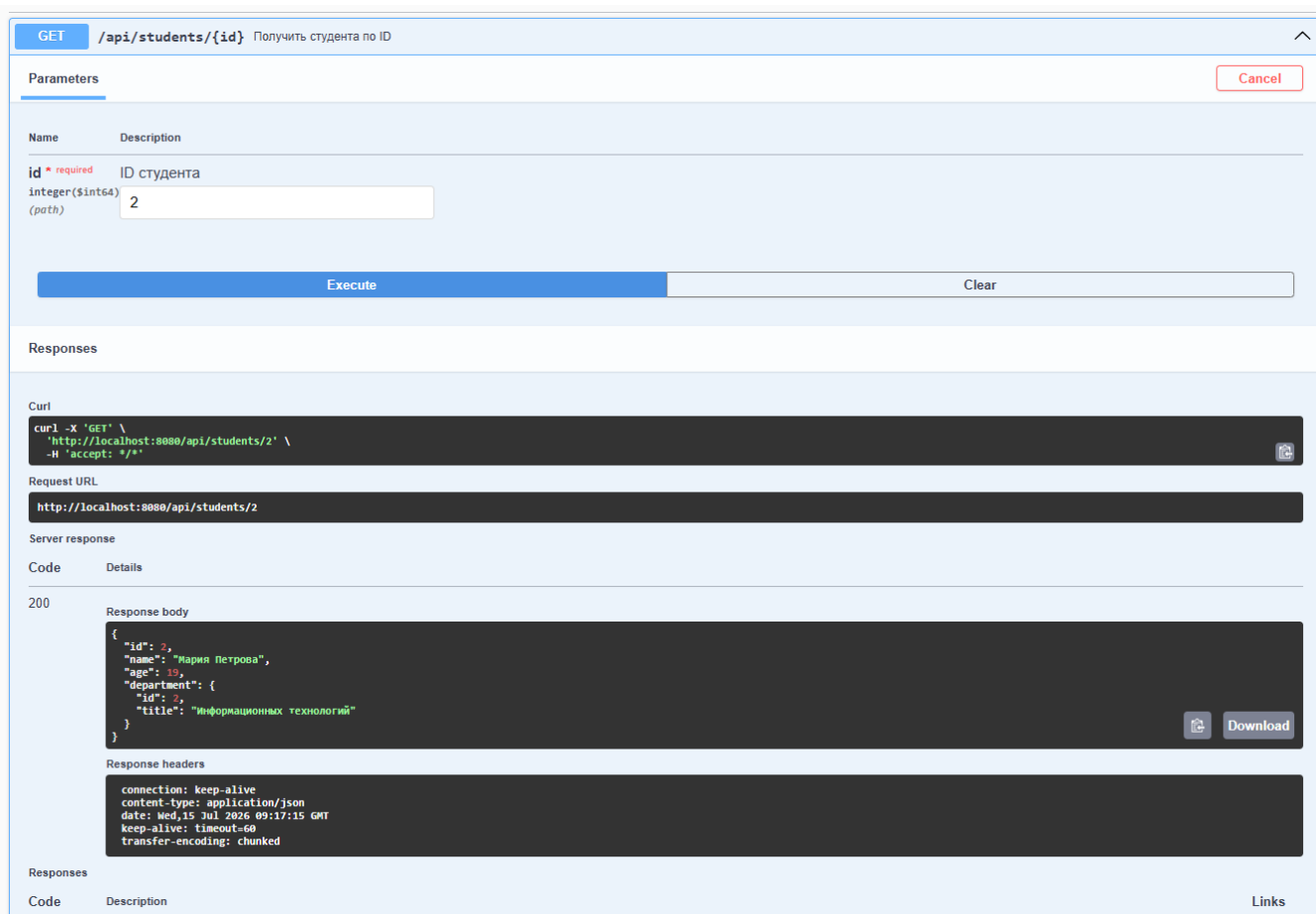


Рисунок 28 – Результат 1

PUT /api/students/{id} Обновить студента

Cancel

Reset

Parameters

Name	Description
id * required integer(\$int64) (path)	ID студента 2

Request body required

application/json

```
{
  "id": 2,
  "name": "Masa",
  "age": 23,
  "departmentId": 1,
  "departmentTitle": "Радиотехнический"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \
  http://localhost:8080/api/students/2 \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 2,
    "name": "Masa",
    "age": 23,
    "departmentId": 1,
    "departmentTitle": "Радиотехнический"
  }'
```

Request URL

```
http://localhost:8080/api/students/2
```

Server response

Рисунок 29 – Результат 2

DELETE /api/students/{id} Удалить студента

Cancel

Parameters

Name	Description
id * required integer(\$int64) (path)	ID студента 1

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
  http://localhost:8080/api/students/1 \
  -H 'accept: */*' \
  -H 'Content-Type: application/json'
```

Request URL

```
http://localhost:8080/api/students/1
```

Server response

Code	Details
204	Response headers <pre>connection: keep-alive date: Wed, 15 Jul 2025 09:29:16 GMT keep-alive: timeout=60</pre>

Responses

Code	Description	Links
204	No Content	No links

Рисунок 30 – Результат 3

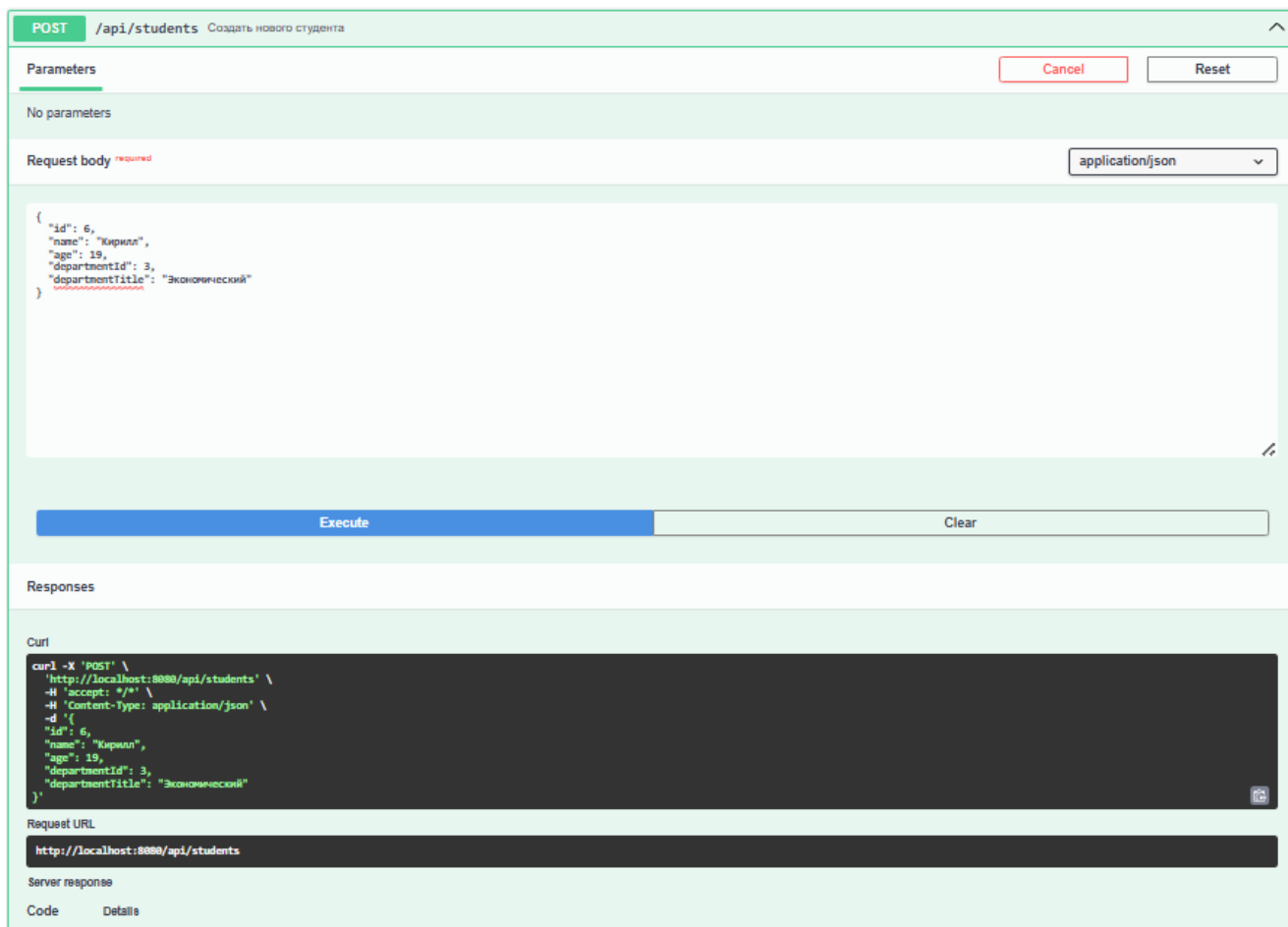


Рисунок 31 – Результат 4

Результаты разработки веб-приложения на Spring Boot представлены на рисунках 32

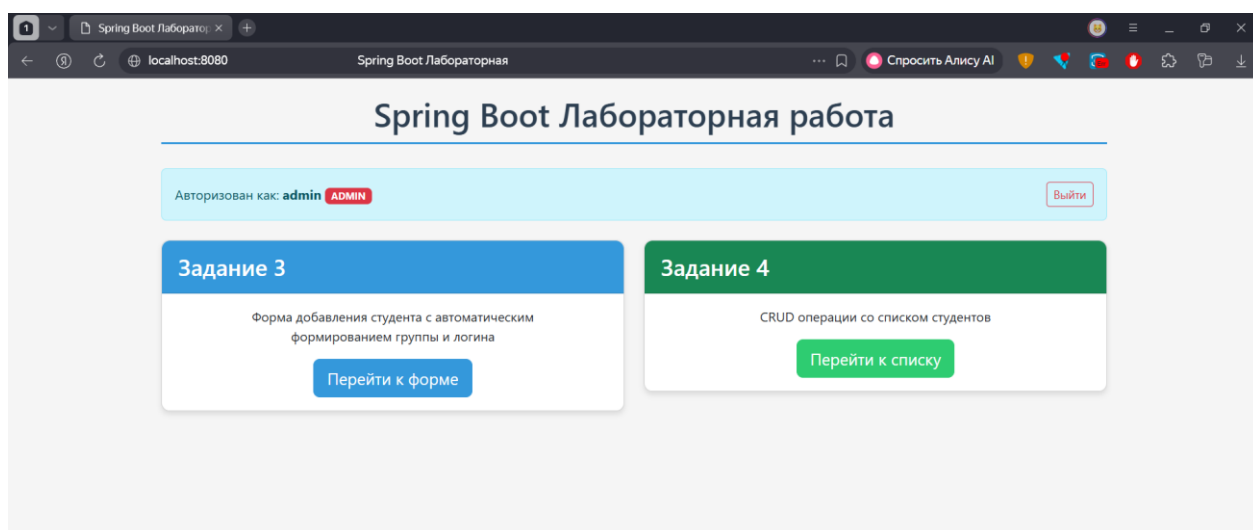


Рисунок 32 – Результат 5

Изм.	Лист	№ докум.	Подпись	Дата

Форма студента - Задание 3

Добавление нового студента

Авторизован как: **admin** ADMIN Выйти

Форма студента (автогенерация группы и логина)

Фамилия Имя Отчество

E-mail Город

Пол Возраст Факультет

Информация: Группа будет сформирована по шаблону "ПИНз-ГГ", где ГГ - последние две цифры текущего года. Логин будет сгенерирован по шаблону "student-{группа}-{id}".

Добавить студента На главную

Рисунок 33 – Результат 6

Список студентов

Авторизован как: **admin** ADMIN Выйти

+ Добавить студента На главную

Поиск студентов

Фамилия Факультет Группа Город Возраст от Найти

ID	ФИО	Группа	Факультет	Город	Действия
1	Иванов Михаил Петрович	РИ-120	Радиотехнический	Москва	Детали Изменить Удалить
2	Петрова Мария Сергеевна	ПИНз-120	Информационных технологий	Санкт-Петербург	Детали Изменить Удалить
3	Алиев Магомед Расулович	РИ-121	Радиотехнический	Махачкала	Детали Изменить Удалить
4	Смирнова Анна Алексеевна	ПИНз-121	Информационных технологий	Владимир	Детали Изменить Удалить

Рисунок 34 – Результат 7

Результаты интеграции базы данных и Spring Security представлены на рисунках 35-36.

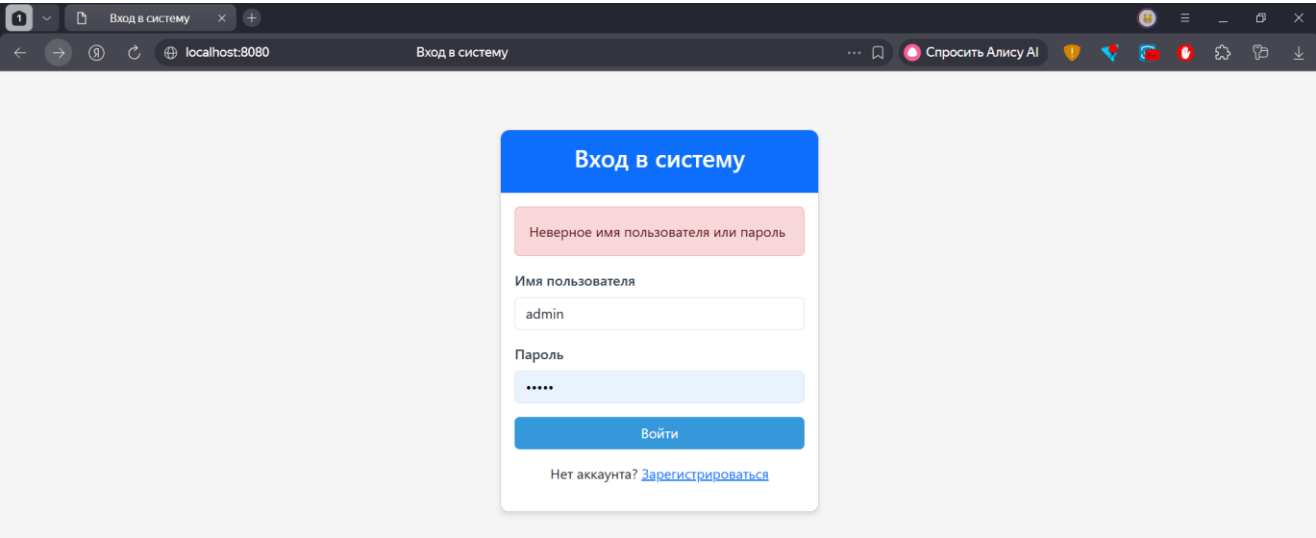


Рисунок 35 – Результат 8

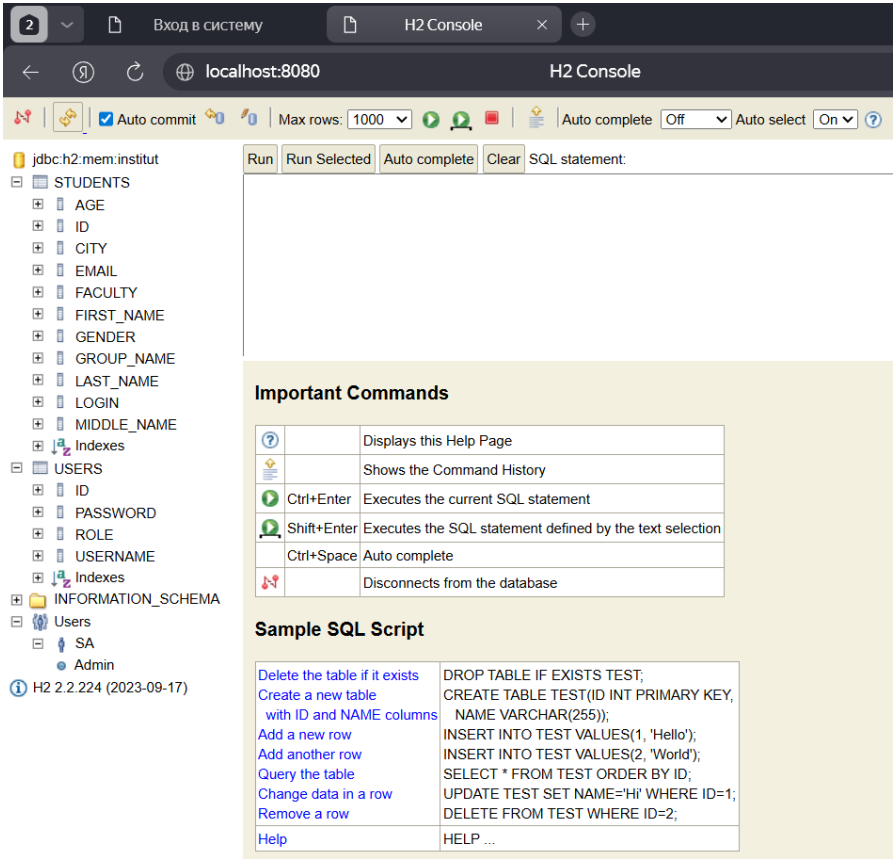


Рисунок 36 – Результат 9

Результат разработки собственного DI-контейнера показан на рисунке 37.

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:D:\games\IntelliJ IDEA Community Edition 2025.2.1\lib\idea_rt.jar=51436" -Dfile.encoding=UTF-8
=== Запуск DI-контейнера ===

Сканирование пакета...
2026-07-15 12:59:37.061 [main] INFO container.AnnotationContainer - Начало сканирования пакета: services
2026-07-15 12:59:37.074 [main] INFO container.AnnotationContainer - Регистрация бина: services.CheckoutService
2026-07-15 12:59:37.083 [main] INFO container.AnnotationContainer - Регистрация бина: services.OrderService
2026-07-15 12:59:37.087 [main] INFO container.AnnotationContainer - Регистрация бина: services.ShoppingCart
2026-07-15 12:59:37.088 [main] INFO container.AnnotationContainer - Завершено сканирование пакета: services. Найдено бинов: 3

Проверка наличия бинов:
Есть бин 'orderService'? true
Есть бин 'shoppingCart'? true
Есть бин 'checkoutService'? true

Получение бинов (ленивая инициализация):
2026-07-15 12:59:37.089 [main] INFO container.AnnotationContainer - Создание бина 'checkoutService' (класс: services.CheckoutService)
CheckoutService создан (#1)
2026-07-15 12:59:37.095 [main] INFO container.AnnotationContainer - Создание бина 'orderService' (класс: services.OrderService)
OrderService создан (#1)
2026-07-15 12:59:37.095 [main] INFO container.AnnotationContainer - Бин 'orderService' создан и инициализирован
2026-07-15 12:59:37.097 [main] INFO container.AnnotationContainer - Создание бина 'shoppingCart' (класс: services.ShoppingCart)
ShoppingCart создан (#1)
2026-07-15 12:59:37.097 [main] INFO container.AnnotationContainer - Бин 'shoppingCart' создан и инициализирован
2026-07-15 12:59:37.097 [main] INFO container.AnnotationContainer - Бин 'checkoutService' создан и инициализирован

Проверка, что бины - синглтоны:
checkout == checkout2? true

=== Оформление заказа ===
Детали заказа #12345
Товар добавлен в корзину: Ноутбук
Товар добавлен в корзину: Мышь
Заказ оформлен успешно!
=====

2026-07-15 12:59:37.097 [main] INFO container.AnnotationContainer - Закрытие контейнера. Очистка 3 созданных бинов

=== Работа контейнера завершена ===

Process finished with exit code 0
```

Рисунок 37 – Результат 10

Заключение

В ходе выполнения практических заданий были изучены и практически реализованы принципы инверсии управления (Inversion of Control, IoC) и внедрения зависимостей (Dependency Injection, DI) – фундаментальные концепции современной разработки на Java, лежащие в основе многих промышленных фреймворков.

Разработан собственный DI-контейнер AnnotationContainer, который демонстрирует внутреннее устройство и принципы работы IoC-контейнеров. Контейнер поддерживает сканирование пакетов для автоматического обнаружения компонентов, регистрацию бинов с автоматическим или явным именем, ленивую инициализацию (создание бинов только при первом обращении) и два способа внедрения зависимостей – через конструкторы и через поля, с использованием аннотаций и рефлексии. В ходе разработки контейнера были реализованы такие механизмы, как поиск подходящего конструктора, разрешение зависимостей для параметров конструктора, обработка циклических зависимостей и корректная обработка ошибок с выбрасыванием соответствующих исключений. Работа контейнера проверена на примере трёх взаимосвязанных сервисов, что подтвердило его корректное функционирование.

Изучен Spring Context – промышленный IoC-контейнер, и реализованы три способа конфигурации: XML-конфигурация (традиционный подход, до сих пор используемый в легаси-проектах), Java-конфигурация с использованием аннотаций @Configuration и @Bean (типобезопасный и гибкий подход) и конфигурация на основе аннотаций с @Component и @Autowired (наиболее современный и удобный способ). Проведён сравнительный анализ трёх подходов, который показал, что аннотации являются наиболее удобным и широко используемым в современной промышленной разработке способом, обеспечивающим минимальное количество конфигурационного кода при высокой читаемости и типобезопасности.

					МИВУ.09.03.02 – 06.000 ПП	Лист
						37
Изм.	Лист	№ докум.	Подпись	Дата		

Разработано полноценное веб-приложение на Spring Boot с использованием Thymeleaf в качестве шаблонизатора и Bootstrap для стилизации интерфейса. Реализованы две ключевые функции: форма добавления студента с автоматическим формированием названия группы (по шаблону «ПИНз-ГГ», где ГГ – последние две цифры текущего года) и логина для образовательного портала (по шаблону «student-{группа}-{id}»), а также полный набор CRUD-операций над списком студентов, включающий просмотр списка, детальную информацию о студенте, добавление новых записей, редактирование существующих и удаление. Приложение имеет удобный и адаптивный пользовательский интерфейс.

Выполнена интеграция базы данных H2 с использованием JPA и Hibernate. Реализованы JPA-сущности с аннотациями, настроены связи между таблицами, созданы репозитории на основе Spring Data JPA с методами поиска по различным критериям, добавлена валидация данных на уровне моделей. Внедрена система аутентификации и авторизации с Spring Security: созданы сущности для хранения пользователей с ролями (ADMIN и USER), реализован сервис для загрузки пользователей из базы данных, настроена собственная страница входа и выхода. Реализовано разграничение прав доступа как на уровне контроллеров (с помощью аннотации @PreAuthorize), так и на уровне представлений Thymeleaf (с помощью sec:authorize). Пользователи создаются автоматически при первом запуске приложения.

Разработан REST API, предоставляющий полноценный интерфейс для управления студентами и кафедрами. API поддерживает все CRUD-операции, а также расширенные методы поиска по имени, возрасту и кафедре. Для передачи данных используются DTO, что обеспечивает изоляцию внутренней модели данных от внешнего API и предотвращает проблемы с циклическими ссылками при сериализации. Реализована глобальная обработка ошибок с возвратом структурированных JSON-сообщений, добавлена валидация входящих данных. API задоку-

					МИВУ.09.03.02 – 06.000 ПП	Лист
						38
Изм.	Лист	№ докум.	Подпись	Дата		

ментировано с помощью Swagger UI, что позволяет разработчикам легко ознакомиться с его возможностями и протестировать работу методов непосредственно через веб-интерфейс.

Все разработанные приложения и компоненты успешно протестированы и готовы к использованию. В результате выполнения работы получены практические навыки разработки на Spring Framework, глубокое понимание внутренних механизмов IoC-контейнеров и умение создавать промышленные веб-приложения с интеграцией базы данных, системой безопасности, REST API и документацией. Полученные знания и навыки могут быть применены при разработке корпоративных приложений, микросервисных архитектур и современных веб-систем на платформе Java.

					МИВУ.09.03.02 – 06.000 ПП	Лист
						39
Изм.	Лист	№ докум.	Подпись	Дата		

Список используемых источников

1. Спилкэ Л. Spring Security в действии : руководство / Л. Спилкэ ; пер. с англ. В. С. Яценкова. – Москва : ДМК Пресс, 2025. – 480 с. – ISBN 978-5-93700-256-3.
2. Nardone, M. Pro Spring Security : Securing Spring Framework 6 and Boot 3-based Java Applications / M. Nardone, C. Scarioni. – 3rd ed. – Berkeley, CA : Apress, 2024. – 289 p. – ISBN 979-8-8688-0034-4.
3. Урванов, Ф. В. Spring и Spring Boot. Разработка облачных приложений на Java : аспектно-ориентированное программирование, хранение и обновление настроек приложения, модуль Spring Core, фреймворк Spring Security, внедрение зависимостей, упаковка приложений в Java, создание приложений в стиле REST, тестирование Spring-приложений / Ф. В. Урванов. – Санкт-Петербург : БХВ-Петербург, 2025. – 384 с. – ISBN 978-5-9775-2049-2. – (Серия: Профессиональное программирование).
4. Nasslahsen, B. Spring Security : Effectively secure your web apps, RESTful services, cloud apps, and microservice architectures / B. Nasslahsen. – 4th ed. – Birmingham : Packt Publishing, 2024. – 596 p. – ISBN 978-1-83546-011-5.

					МИВУ.09.03.02 – 06.000 ПП	Лист
						40
Изм.	Лист	№ докум.	Подпись	Дата		

